

Prüfung Betriebssysteme

KNr.

MNr.

Zuname, Vorname

Studium:

☐ Informatik (Bakk.)☐ Elektrotechnik (Master)

Ges.)(100)

1.)(30)

2.)(30)

3.)(40)

Zusatzblätter:

Bitte verwenden Sie nur dokumentenechtes Schreibmaterial!**1 Synchronisation mit Semaphoren (30)**

Ein Softwarepaket aus vier Prozessen soll eine Anlage simulieren, die Kartons mit Würfeln und Murmeln befüllt. Zwei gleichartige Prozesse simulieren den An- und Abtransport von Kartons zur bzw. von der Befüllanlage. Je ein Prozess kontrolliert das Einfüllen von Murmeln bzw. Würfeln in einen bereitstehenden Karton.

Ergänzen Sie die gegebenen Codestücke mit *Semaphoroperationen*, um die Prozesse entsprechend der angeführten Bedingungen zu synchronisieren. Das Verwenden von globalen Variablen zur Synchronisation oder Kommunikation zwischen Prozessen ist nicht erlaubt.

- Die beiden Prozesse für den An- und Abtransport der Kartons sollen Roboter simulieren. Jeder Roboter hat einen eigenen Vorrat an leeren Kartons und einen eigenen Ablageplatz für volle Kartons und führt wiederholt folgende Aktionen durch:
 - Karton vom Vorrat holen: *karton_aufnehmen()*.
 - Karton zur Befüllanlage transportieren und öffnen: *karton_bereitstellen()*.
 - Karton verschließen und von der Befüllanlage nehmen: *karton_entfernen()*.
 - Karton zum Ablageplatz abtransportieren: *karton_abtransportieren()*.
- Der Prozess Murmelspender ruft wiederholt die Funktion *Murmel_liefern()* auf. Jeder Aufruf der Funktion lässt eine Murmel in den Karton auf der Befüllanlage fallen.
- Der Prozess Würfelspender ruft wiederholt die Funktion *Wuerfel_liefern()* auf. Jeder Aufruf der Funktion lässt einen Würfel in den Karton auf der Befüllanlage fallen.
- Die Prozesse sind so zu synchronisieren, dass die Parallelität ihrer Aktionen die folgenden Regeln erfüllt, darüberhinaus aber möglichst wenig eingeschränkt wird.
- Auf dem Befüllplatz der Anlage ist Platz für einen Karton. Die Reihenfolge, in der die Roboter die Befüllanlage mit Kartons versorgen, ist nicht einzuschränken.
- Jeder Karton ist mit N Murmeln und $N + K$ Würfeln zu befüllen. Dabei sollen zuerst N Murmeln bzw. Würfel abwechselnd, beginnend mit einem Würfel, und dann die restlichen K Würfel in die Kartons gefüllt werden.

(a) Initialisierungen (vor Start der Prozesse)

```
init(k, 1);  
init(W, 0);  
init(M, 0);  
init(x, 1);  
init(y, 0);  
init(z, 0);
```

In der Folge sind Codestücke für die zu synchronisierenden Prozesse gegeben, die bereits alle notwendigen Funktionsaufrufe enthalten. Ergänzen Sie in diesen Codestücken die fehlenden Semaphoroperationen zur Synchronisation.

(b) Prozesse für die Befüllung

```
/** Murmelspender **/
```

```
do {
```

```
wait(M);  
wait(z);
```

```
Murmel_liefern();
```

```
signal(y);
```

```
} while(1);
```

```
/** Wuerfelspender **/
```

```
do {
```

```
wait(W);
```

```
Wuerfel_liefern();
```

```
signal(z);  
wait(y);  
signal(x);
```

```
wait(W);
```

```
Wuerfel_liefern();
```

```
signal(z);  
wait(y);  
signal(x);
```

```
} while(1);
```

(c) Prozesse für Roboter

```
/** Roboter R1 **/
```

```
do {
```

```
    karton_aufnehmen(R1);
```

```
    wait (K);
```

```
    karton_bereitstellen(R1);
```

```
    for (int i=0; i<N; i++) {  
        signal(W);  
        signal(M);  
        wait(x);  
    }  
    for (int i=0; i<K; i++) {  
        signal(W);  
        signal(y);  
        wait(z);  
        wait(x);  
    }  
}
```

```
    karton_entfernen(R1);
```

```
    signal(x);  
    signal(K);
```

```
    karton_abtransportieren(R1);
```

```
} while (1)
```

```
/** Roboter R2 **/
```

```
do {
```

```
    karton_aufnehmen(R2);
```

```
    wait (K);
```

```
    karton_bereitstellen(R2);
```

```
    for (int i=0; i<N; i++) {  
        signal(W);  
        signal(M);  
        wait(x);  
    }  
    for (int i=0; i<K; i++) {  
        signal(W);  
        signal(y);  
        wait(z);  
        wait(x);  
    }  
}
```

```
    karton_entfernen(R2);
```

```
    signal(x);  
    signal(K);
```

```
    karton_abtransportieren(R2);
```

```
} while(1);
```

2 Scheduling (30)

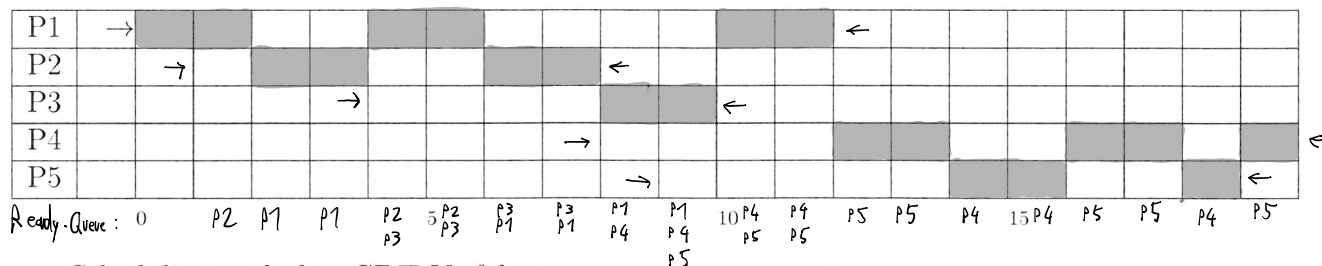
Schedulen Sie das nebenstehende Taskset mit den Verfahren *Round Robin* (RR), *Shortest Remaining Time* (SRT), bzw. *Highest Response Ratio Next* (HRRN). Der Overhead für den Taskwechsel ist vernachlässigbar. Bei *RR* beträgt die Zeitscheibenlänge 2 Zeiteinheiten.

Task	Arrival Time	Service Time
P1	0	6
P2	1	4
P3	4	2
P4	8	5
P5	9	3

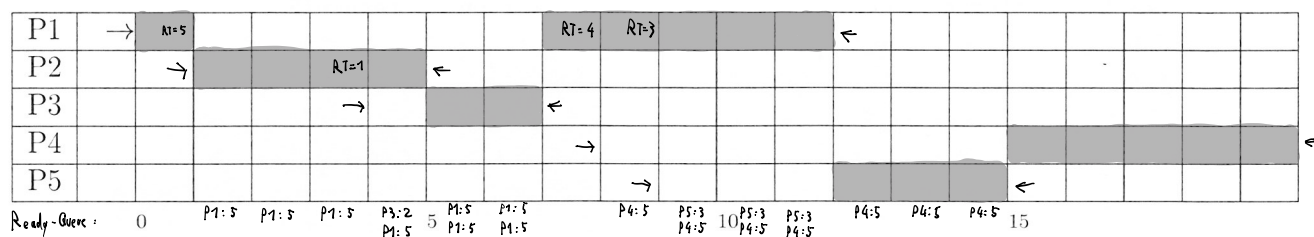
Fallen beim *RR*-Scheduling das Ende einer Zeitscheibe und die Ankunft eines Tasks zeitlich zusammen, so behandeln Sie den unterbrochenen Task vor dem neuen. Bei *SRT* bzw. *HRRN* ist zu beachten, dass Tasks zu ihrem Ankunftszeitpunkt sofort gescheduled werden können. Gibt es zu einem Zeitpunkt mehrere Tasks mit höchster Priorität, so soll der Task mit der niedrigsten Nummer ausgeführt werden (also beispielsweise *P4* vor *P5*).

Markieren Sie in den nachstehenden Vorlagen zu jedem Zeitpunkt den jeweils aktiven Task. Kennzeichnen Sie die *Arrival Time* jedes Tasks mit einem Pfeil “ $\rightarrow$ ” und den Zeitpunkt, zu dem ein Task seine gesamte *Service Time* konsumiert hat, mit “ $\leftarrow$ ”. Eine Vorlage dient als Ersatz. Streichen Sie gegebenenfalls die falsch ausgefüllte Vorlage deutlich durch.

Scheduling nach dem **RR**-Verfahren:



Scheduling nach dem **SRT**-Verfahren:



Scheduling nach dem **HRRN**-Verfahren:

